

Object Oriented Programming Concepts

- class
- object
- encapsulation
- interface
- inheritance
- abstraction
- polymorphism

```
class Main{
    public static void main(String[] a){
        Integer x = 10;
        System.out.println(a);
    }
}
class Test{
    Integer a = 20;
}
```

A.java

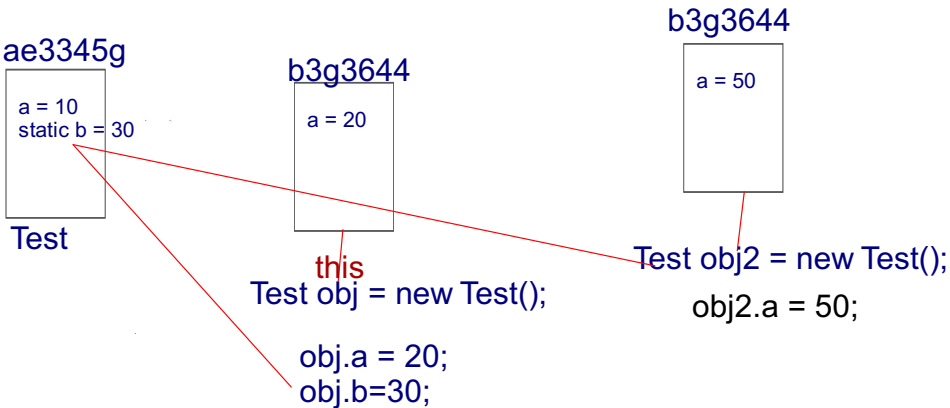
compilation: javac A.java



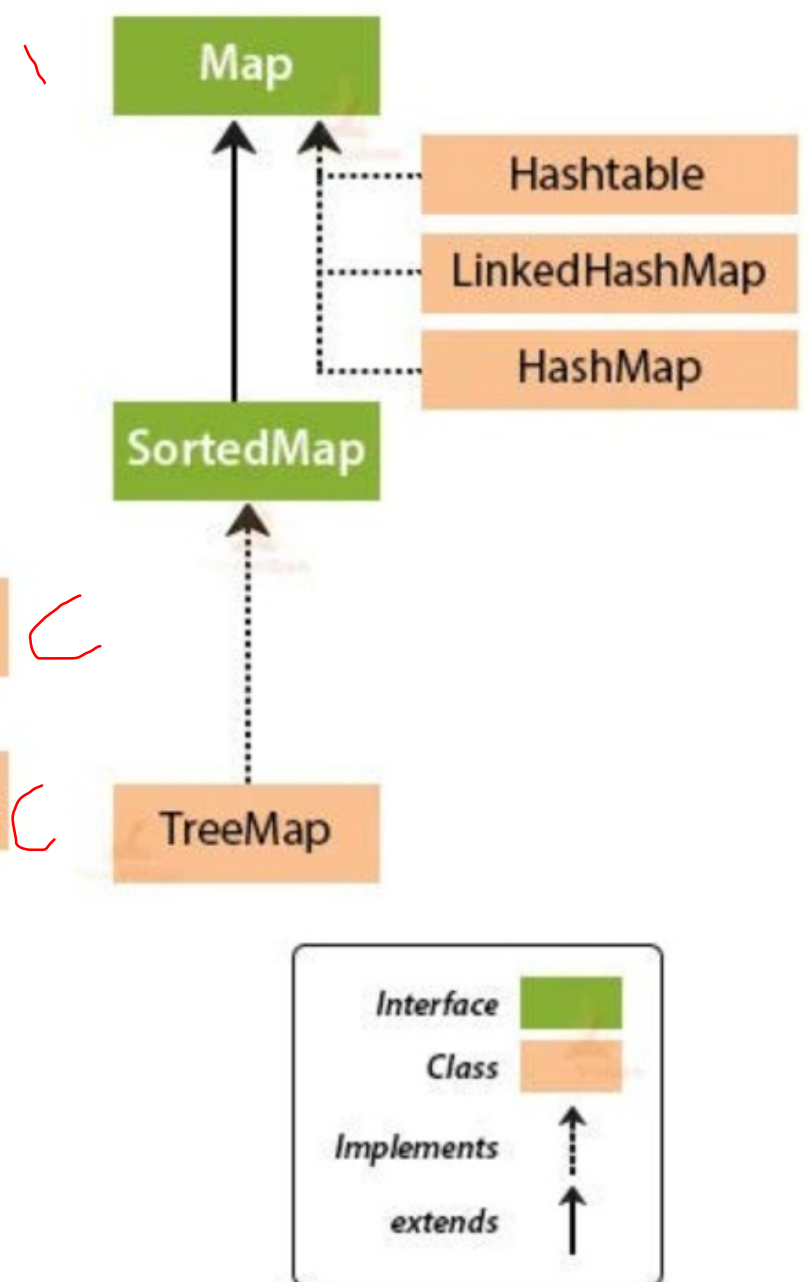
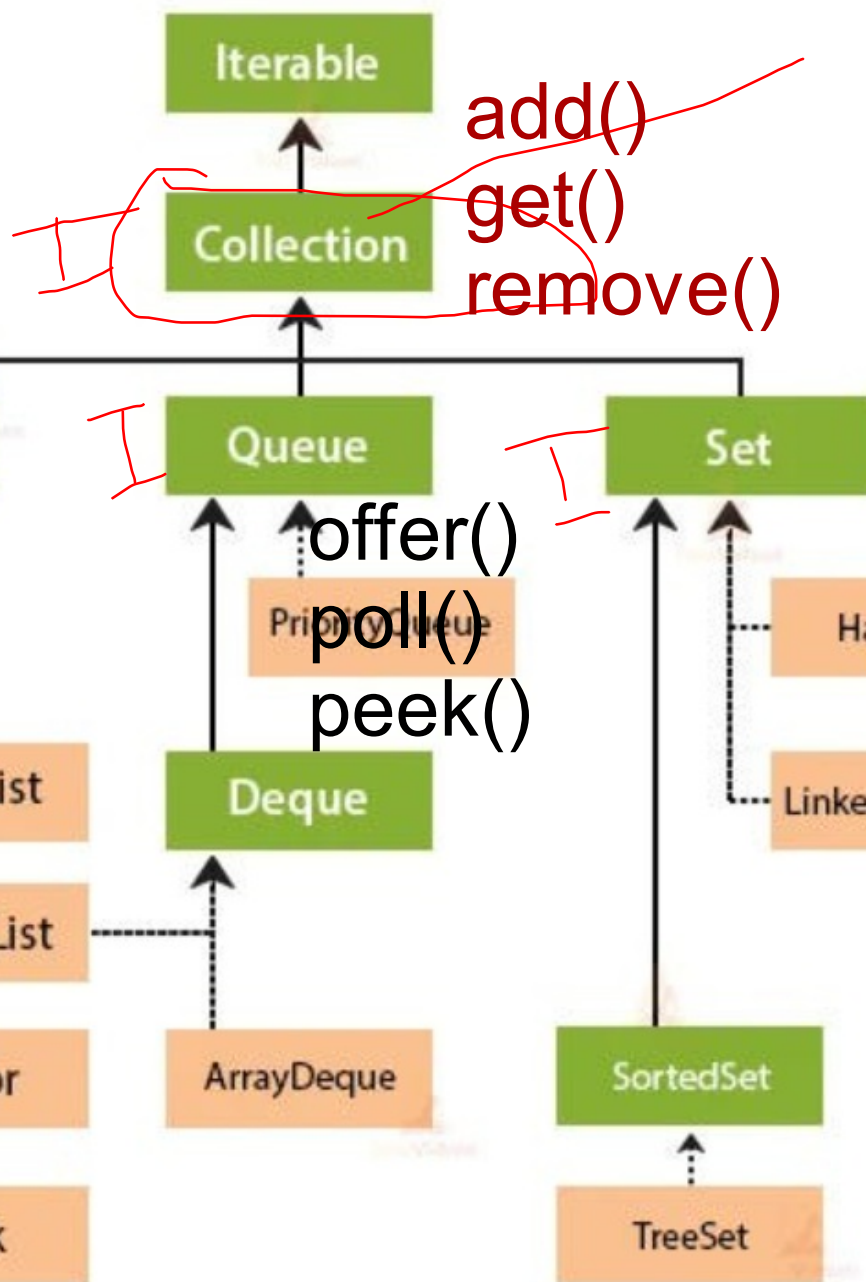
Main.class, Test.class (Byte Code)

interpretation: java Main

Output



add(int index)
addFirst();
addLast();
get(index);
remove(index);
push();
pop();
peek();



UPI

Card

IB

COD

payment(amount)

Payment Modes

Packages

com.bit.model -> encapsulated classes

com.bit.service -> service interfaces

com.bit.dao -> service implementation classes

com.bit.controller -> Main class

Java Database Connectivity

Library
Java MySQL Connector



Prepare Statement
Execute Query
ResultSet
Close Connection

Naming Conventions

class/interface	PascalCase	Payment, PaymentService
field/method	camelCase	name, firstName, display(), readInput()
constants	UPPER_SNAKE_CASE	PIE, PERSONAL_LOAN_INTEREST
packages	lower case with dots	edu.bit.training

SOLID Principles

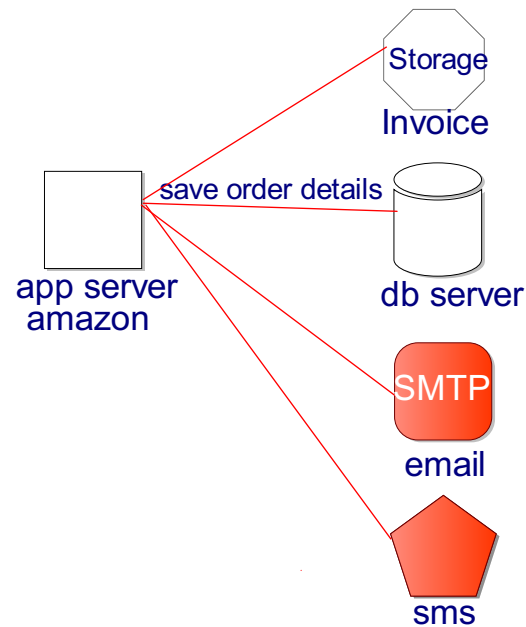
Single Responsibility Principle (SRP)

Open/Closed Principle (OCP)

Liskov Substitution Principle (LSP)

Interface Segregation Principle (ISP)

Dependency Injection Principle (DIP)



```
class OrderServiceImpl{
    //code to generate invoice
    //code to save in database
    //code to send mail with invoice
    //code to send sms with order info
}
```

After SOLID Principles - SRP

```
interface NotificationService{}
class NotificationServiceImpl implements NotificationService{}
interface InvoiceService{}
class InvoiceServiceImpl implements InvoiceService{}
interface OrderService{}
class OrderServiceImpl implements OrderService{
    //invoice service injection
    InvoiceService is = new InvoiceServiceImpl();
    NotificationService ns = new NotificationServiceImpl();
    //code to save order into database
}
```